

LIBRARY MANAGEMENT SYSTEM

Build Your Own Project

CSE2006 – Programming in Java

Submitted by

Lav Singh Gautam

Project Type

Command-Line Java Application

Technologies Used

Java • Object-Oriented Programming • Collections • File I/O • Git
& GitHub

VITyarthi – Build Your Own Project

Academic Year: 2026

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Structure
8. Design Decisions and Rationale
9. Implementation Details
10. Object-Oriented Programming Concepts Used
11. Module Description
12. Data Storage and File Persistence
13. Screenshots and Results
14. Testing Approach and Results
15. Challenges Faced
16. Learnings and Key Takeaways
17. Future Enhancements
18. Conclusion
19. References

1. INTRODUCTION

1.1 Overview

The Library Management System is a command-line-based Java application developed to manage basic library operations in a structured manner. The system provides different functionalities for administrators and library members. Administrators can manage books and view registered users and transaction history, while members can search for books, borrow books, return books, and view their profiles. The project demonstrates the practical application of Java and Object-Oriented Programming concepts in a real-world problem. The application uses a modular architecture consisting of models, services, interfaces, utilities, exception handling, and test components.

1.2 Purpose

The purpose of the project is to create a simple library management solution and demonstrate how Java programming concepts can be combined to develop a modular, maintainable application. The system focuses on User Management, Book Management, and Transaction Management. The application also uses file-based persistence to store book information, ensuring book records remain available when the application is restarted.

2. PROBLEM STATEMENT

Traditional manual library management can become difficult when the number of books and users increases. Maintaining book records, checking availability, managing borrowing and returning operations, and tracking transactions manually can result in

inefficient record management and errors. A computerized system is therefore required to organize these operations and provide a structured way to manage library resources. The proposed Library Management System provides a command-line solution that allows administrators to manage books and users while allowing members to search, borrow, and return books. The project also demonstrates how Object-Oriented Programming concepts can be applied to model real-world entities such as books, users, and transactions.

3. OBJECTIVES

The main objectives of the project are:

1. To develop a functional Library Management System using Java.
2. To implement a role-based system for administrators and members.
3. To provide functionality for adding, removing, searching, and viewing books.
4. To implement book borrowing and returning operations.
5. To enforce borrowing limits according to membership type.
6. To calculate overdue fines based on the borrowing period.
7. To maintain book availability information.
8. To demonstrate abstraction, encapsulation, inheritance, and polymorphism.
9. To demonstrate interfaces and generics.
10. To implement file input/output for persistent book storage.
11. To organize the application into separate models, services, utilities, and other components.

12. To test important system operations using automated Java tests.

4. FUNCTIONAL REQUIREMENTS

The system contains three major functional modules with clear inputs, outputs, and a logical workflow.

4.1 User Management

The User Management module handles different types of users.

- Administrator login
- Member login
- Role-based menu access
- Member membership types
- User profile display
- Password verification
- Duplicate email prevention

4.2 Book Management

The Book Management module handles the library's book records.

- Add books
- Remove books
- View all books
- Search books
- Find books using their ID
- Check book availability
- Prevent duplicate book IDs
- Save book information to a file
- Load book information when the application starts

4.3 Transaction Management

The Transaction Management module handles borrowing and returning operations.

- Borrow a book
- Return a book
- Check book availability
- Enforce member borrowing limits
- Maintain transaction information
- Calculate overdue fines
- Display transaction history

The system supports two types of books: Physical Book and E-Book. Regular members can borrow up to 3 books, while Premium members can borrow up to 7 books. The standard borrowing period is 14 days, and the overdue fine is ₹2 per day after the allowed period.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Usability

The application uses a simple command-line interface with numbered menus. Users can select operations by entering numeric choices.

5.2 Reliability

The system validates important operations. A book cannot be borrowed if it is unavailable; a member cannot exceed their borrowing limit; duplicate book IDs are rejected, and invalid numeric input is handled.

5.3 Maintainability

The project is divided into models, services, interfaces, utilities, and exceptions, making individual components easier to understand and modify.

5.4 Resource Efficiency

The application uses Java collections for in-memory data management and simple file-based persistence without requiring a database server.

5.5 Error Handling

The application validates invalid menu choices, numeric input, duplicate book IDs, unavailable or missing books, and invalid login credentials. A custom `LibraryException` class is also included.

6. SYSTEM ARCHITECTURE

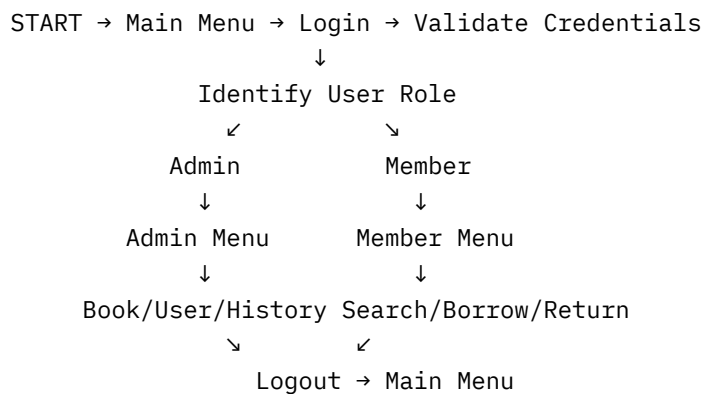
The application follows a modular architecture. `Main.java` handles the command-line interface and workflow. Model classes represent books, users, and transactions. Service classes contain application logic. Utility classes handle file operations and reporting abstractions. The data layer contains the persistent `books.dat` file.

7. DESIGN Structure

7.1 Use Case Design

Actors: Administrator and Member. Administrator operations include login, viewing books, adding books, removing books, viewing users, and viewing transaction history. Member operations include logging in, viewing books, searching books, borrowing books, returning books, and viewing the profile.

7.2 Workflow Diagram



7.3 Class Design

Book is the parent of PhysicalBook and EBook. User is the parent of Admin and Member. BookService, UserService, and TransactionService provide operations over these models.

7.4 Storage Design

The project uses flat-file storage rather than a relational database. The primary persistent file is data/books.dat.

8. DESIGN DECISIONS AND RATIONALE

8.1 Abstract Classes

Book and User are abstract classes because they represent common characteristics shared by different types of books and users.

8.2 Inheritance

Inheritance reduces code duplication and allows specialized classes to reuse common functionality.

8.3 Interfaces

Searchable <T> defines a search contract, and BookService implements Searchable <Book>. The project also includes Reportable as a reporting abstraction.

8.4 Collections

ArrayList is used to store books, users, transactions, and borrowed book IDs.

8.5 File-Based Storage

A text file was selected because the project is a lightweight command-line Java application and does not require a database server for its current scope.

9. IMPLEMENTATION DETAILS

9.1 Main Class

Main.java is the entry point. It initializes services, loads saved books, displays the main menu, handles login, and directs users to role-specific menus.

9.2 Book Classes

The abstract Book class stores common book information. PhysicalBook adds shelf location, while EBook adds file size and format. Polymorphism is demonstrated through overridden getBookType() and displayDetails().

9.3 User Classes

The abstract User class stores common user information. Admin and Member inherit from User. displayProfile() is overridden by both classes.

9.4 BookService

BookService manages books by adding, removing, finding, searching, displaying, loading, and saving records.

9.5 UserService

UserService manages users, email-based lookup, login authentication, profile display, and default user initialization.

9.6 TransactionService

TransactionService handles borrowing, returning, availability updates, borrowing limits, and transaction history.

9.7 Transaction Class

Transaction stores transaction ID, member ID, book ID, borrow date, return date, return status, and fine calculation logic.

10. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

Concept	Implementation
Abstraction	Book and User abstract classes
Encapsulation	Private fields with controlled access through methods

Inheritance	PhysicalBook, EBook, Admin, and Member
Polymorphism	Overridden displayDetails(), displayProfile(), getRole(), and getBookType()
Interfaces	Searchable<T> and Reportable
Generics	Searchable<Book>
Collections	ArrayList and List
Exception Handling	LibraryException and file-operation exception handling
File I/O	FileHandler using BufferedReader and BufferedWriter
Method Overriding	Subclasses override parent behavior

11. MODULE DESCRIPTION

Module 1 – User Management

This module manages administrators and members.

Main classes:

- User
- Admin
- Member
- UserService

Module 2 – Book Management

This module manages the library's books.

Main classes:

- Book
- PhysicalBook
- EBook

- BookService

Module 3 – Transaction Management

This module handles library transactions.

Main classes:

- Transaction
- TransactionService

12. DATA STORAGE AND FILE PERSISTENCE

The application uses data/books.dat. FileHandler provides file input/output using BufferedReader, BufferedWriter, FileReader, and FileWriter. Book records are stored using pipe-separated values.

```
ID|Title|Author|PHYSICAL|Shelf|Availability
```

```
ID|Title|Author|EBOOK|FileSize|Format|Availability
```

When the application starts, BookService loads existing records. When a book is added or removed, the updated collection is written back to the file.

13. SCREENSHOTS AND RESULTS

13.1 Main Menu

```
User added successfully.  
User added successfully.  
User added successfully.  
=====
      LIBRARY MANAGEMENT SYSTEM
=====

1. Login
2. View All Books
3. Search Books
4. Exit
Enter choice: █
```

Demonstrates the main command-line interface.

13.2 Member Login

```
=====
      LIBRARY MANAGEMENT SYSTEM
=====

1. Login
2. View All Books
3. Search Books
4. Exit
Enter choice: 1
Email: john@library.com
Password: john123

Login successful.
Role: Member
Name: John Member
Email: john@library.com
Membership: REGULAR
Borrowed Books: 0

===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: █
```

Demonstrates successful member authentication.

13.3 Member Menu

```
===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: █
```

Demonstrates operations available to a member.

13.4 Book Search

```
===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: 2
Enter title or author keyword: Java

Search Results:
ID: 1
Title: Java
Author: Author
Type: Physical Book
Available: true
Shelf: A-101
-----
```

Demonstrates searching using a title or author keyword.

13.5 Borrow Book

```
===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: 3
Enter book ID to borrow: Java
Please enter a valid number.
Enter book ID to borrow: 1
Book borrowed successfully.
```

Demonstrates a successful borrowing operation.

13.6 Return Book

```
===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: 4
Enter book ID to return: 1
Book returned successfully.
Fine: ?0
```

Demonstrates returning a borrowed book and displaying the fine.

13.7 Admin Menu

```
===== MEMBER MENU =====
1. View Books
2. Search Books
3. Borrow Book
4. Return Book
5. View My Profile
6. Logout
Enter choice: 6

1. Login
2. View All Books
3. Search Books
4. Exit
Enter choice: 1
Email: admin@library.com
Password: admin123

Login successful.
Role: Admin
Name: Library Admin
Email: admin@library.com

===== ADMIN MENU =====
1. View Books
2. Add Book
3. Remove Book
4. View Users
5. View Transaction History
6. Logout
```

Demonstrates administrator-specific functionality.

13.8 Transaction History

```
===== ADMIN MENU =====
1. View Books
2. Add Book
3. Remove Book
4. View Users
5. View Transaction History
6. Logout
Enter choice: 5
Transaction ID: 1 | Member ID: 2 | Book ID: 1 | Borrowed: 2026-09-17 | Returned: true
Return Date: 2026-09-17
Fine: ?0
-----
```

Demonstrates transaction records.

14. TESTING APPROACH AND RESULTS

Testing was performed using tests/LibraryTest.java. The test suite checks important system operations.

Test	Result
Add and Find Book	PASS
Duplicate Book ID Prevention	PASS
Book Search	PASS
Borrow Book	PASS
Return Book	PASS
Regular Member Borrowing Limit	PASS

Overall Result: Tests Passed: 6 | Tests Failed: 0. The complete test suite passed successfully.

15. CHALLENGES FACED

15.1 Class Structure

A challenge was deciding how real-world entities such as books and users should be represented using Java classes. This was addressed using abstract base classes and specialized subclasses.

15.2 Polymorphism

Different types of books and users needed different display behavior while sharing common functionality. Method overriding was used.

15.3 File Persistence

Maintaining book information after the application was closed required persistent storage. `FileHandler` was implemented for this purpose.

15.4 Input Handling

Command-line applications can encounter invalid user input. Input validation was added for numeric values and menu choices.

15.5 Transaction Validation

Borrowing and returning require several conditions to be checked. `TransactionService` validates availability, borrowing limits, and active transactions.

16. LEARNINGS AND KEY TAKEAWAYS

13. Abstract classes and abstraction
14. Inheritance and parent-child class relationships
15. Polymorphism through method overriding
16. Encapsulation using private fields and methods
17. Interfaces and generics
18. `ArrayList` and `List` collections
19. Java file input/output
20. Package-based project organization
21. Error and input handling
22. Automated Java testing

23. Git and GitHub version control

17. FUTURE ENHANCEMENTS

17.1 Database Integration

Replace text-file storage with a relational database such as MySQL or PostgreSQL.

17.2 User Persistence

Store user information permanently instead of initializing default users.

17.3 Transaction Persistence

Store transaction records permanently.

17.4 Graphical User Interface

Develop a GUI using JavaFX or Swing.

17.5 Enhanced Security

Use secure password hashing.

17.6 Advanced Search

Add filters for book type, author, availability, category, and publication year.

17.7 Automated Reporting

Generate reports about borrowed books, active transactions, overdue books, and member activity.

18. CONCLUSION

The Library Management System demonstrates how Java and Object-Oriented Programming concepts can be applied to solve a practical

problem. The system provides three major functional areas: User Management, Book Management, and Transaction Management. It allows administrators to manage library resources and members to search, borrow, and return books. The project demonstrates abstraction, encapsulation, inheritance, polymorphism, interfaces, generics, collections, exception handling, and file input/output. The application was organized into separate packages and tested using a dedicated Java test suite. All six implemented tests passed successfully. The project also uses Git and GitHub for version control and project management.

19. REFERENCES

24. VITyarthi, Build Your Own Project – General Project Instructions & Submission Guidelines, CSE2006 – Programming in Java.
25. Oracle, Java Documentation, Java Platform documentation.
26. Oracle, The Java Tutorials – Object-Oriented Programming Concepts.
27. Oracle, Java Collections Framework Documentation.
28. Oracle, Java I/O Documentation.
29. GitHub, GitHub Documentation.
30. Project source code and documentation are maintained in the project's GitHub repository.